

SUMMARY

Applied AI systems builder focused on making AI-assisted software work reliable. I design workflows where coding agents read the right context, work on one clear task, change the right files, run checks, and leave notes the next session can use.

Background in Unity/C# systems, UEFN/Verse, simulation tooling, and production pipelines. My portfolio shows real examples of AI-assisted engineering, runtime systems, debugging, validation, and working prototypes.

CORE SKILLS

AI Workflow Design: clear tasks, project rules, context loading, repeatable agent work

Agent Control: keep AI focused, prevent wrong-file edits, protect generated assets

Validation: build checks, data checks, runtime logs, screenshots, bug evidence

Developer Tooling: Unity editor tools, import/build tools, prompt helpers, debug surfaces

Runtime Systems: simulations, game logic, stats, conditions, AI behavior

Delivery: turn vague ideas into focused plans, prototypes, and usable V1 systems

PROOF POINTS

Workflow case study: controlled AI coding process

AI systems demo: prompts become checked, working behavior

Unity portfolio: proof videos for tools, simulation, rendering, pathfinding

Reusable framework: rules, logs, checks, saves, debug tools

Multiple prototypes: agent workflow tested across real projects

TECHNICAL STACK

{} Languages
C#, JavaScript, HTML, CSS, Python scripting, Verse

AI AI Tools
Codex, ChatGPT, Claude, Cursor, Gemini, agent-assisted implementation workflows

SYS Systems
Unity architecture, runtime simulation, data-driven systems, pathfinding, procedural generation, editor tooling

OPS Production
Git, JSON schemas, validation profiles, logs, screenshots, 3ds Max, Maya, ZBrush, Photoshop

EDUCATION

BFA, Media Arts and Animation
Art Institute of Seattle

UNIQUE PERSPECTIVE

My art and game development background is a strength for AI systems work. Moving between art, design, and engineering taught me to speak multiple team languages and build tools people can actually use: readable state, feedback loops, debugging, and iteration. That helps me make AI workflows understandable, controllable, and trusted.

SELECTED WORK



Controlled AI Coding Workflow
ryansharr.com

CONTROLLED AI ENGINEERING

Public case study showing how I make coding agents like Codex useful on real projects instead of treating them like chatbots.

- Built project instructions so the coding agent knows what to read, what rules to follow, and when work is done
- Turns prompts into written tasks with clear files, checks, and reviewable changes
- Protects generated files by changing the source tools instead of fragile generated output
- Uses build checks, data checks, runtime logs, screenshots, and notes to prove fixes worked
- Leaves project notes so future AI sessions and humans can continue without rediscovering context



AI Ideas to Working Software

- AI ideas become structured data, checked inputs, generated assets, and working Unity behavior.
- Runtime code owns the actual state and rules instead of trusting generated text.
- Debug tools show what changed so problems can be traced and fixed.
- Custom tools handle import, checks, build steps, updates, and proof that the system works.



Unity Systems Proof

- Built reusable systems for stats, conditions, behavior, traversal, and simulation.
- Built pathfinding, procedural movement, editor tools, and generated asset workflows.
- Portfolio videos show functioning systems, not mockups or static screenshots.
- Game-engine work translates directly to complex state, tools, debugging, and product systems.

EXPERIENCE

Independent AI Workflow and Systems Development

Self-directed

Built AI-assisted workflows, Unity systems, runtime prototypes, and public portfolio pages focused on turning ambiguous ideas into governed, working software.

- Designed controlled Codex workflows that turn prompts into clear tasks with rules, checks, logs, and reviewable output.
- Built reusable Unity infrastructure for debug tools, runtime logs, saving, time, input, and validation.
- Created authoring pipelines where generated scenes, data files, narrative content, and runtime connections are checked before use.
- Used runtime traces to diagnose physics, movement, UI, and asset issues, then fixed the source systems.
- Published portfolio pages explaining the workflow, AI system architecture, proof videos, and debugging case studies.

Engineering Lead / Creative Director

Loot Foundry Ltd

Led engineering and creative direction for system-driven gameplay development in UEFN and Verse, bridging design, art, and implementation under production constraints.

- Built Verse gameplay systems including stats, conditions, buffs, debuffs, and state-driven behavior.
- Designed centralized runtime architecture decoupled from scene objects and reusable across gameplay systems.
- Translated ambiguous design goals into structured technical implementation and reviewable work items.
- Directed system design across gameplay, UX, technical execution, debugging, and workflow decisions.
- Served as technical point of contact for cross-discipline implementation and production blockers.

3D Character Artist

Gas Powered Games

Production art role that built the foundation for pipeline discipline, technical constraints, and cross-discipline game development.

- Modeled and textured production-ready character assets from concept art.
- Collaborated with animation, VFX, and design for gameplay integration.
- Worked within production pipelines under technical and performance constraints.